Computational Statistics
Unit 4: Stochastic Simulation

Nicolás Rivera

Universidad de Valparaíso
CHILE

IDEUV

1. Introduction to Stochastic Simulation

Basics of Stochastic Simulation

The goal of Stochastic Simulation is to simulate random variables and stochastic processes based on a sequence $(U_i)_{i\geq 1}$ of uniform random variables in $[0, 1]$

In Nympy we obtain a vector of n independent $\mathcal{U}(0, 1)$ random variables $(U_i)_{i=1}^{n}$ by

```
U = np.random.uniform(0,1,n)
print(n)
```

Two important distributions are the following

$\mathcal{U}(a, b)$

Given $a < b$, the $\mathcal{U}(a, b)$ distribution is a continuous distribution with density $1/(b - a)$ in the interval $[a, b]$.
Given $a < b$ and $U \sim \mathcal{U}(0, 1)$, then

$$U' = a + U(b - a)$$

is such that $U' \sim \mathcal{U}(a, b)$. In Python: `np.random.uniform(a,b,n)`.

$\mathcal{U}(\{1, \ldots, K\})$

The $\mathcal{U}(\{1, \ldots, K\})$ is a discrete distribution that mass $1/K$ to each element of $\{1, \ldots, K\}$.

$$U' = \lfloor kU \rfloor + 1,$$

where $U \sim \mathcal{U}(0, 1)$. In Python just do `random.randint(1, K,n)`.

Exercise: Let $U \sim \mathcal{U}(0, 1)$. Show that

1. If $a < b$, then $U' = a + U(b - a)$ has distribution $\mathcal{U}(a, b)$.
2. If $K > 0$ is an integer, then $U' = \lfloor kU \rfloor + 1$ is $\mathcal{U}(\{1, 2, \ldots, K\})$

2. Simulation of Discrete Random Variables

Simulation of Discrete Random variables

We will study the following four tools to simulate from discrete random variables

1. Linear search sampling
2. Binary search sampling
3. Sampling of Combinatorial Objects: Random Permutations and Random subsets

2. Simulation of Discrete Random Variables

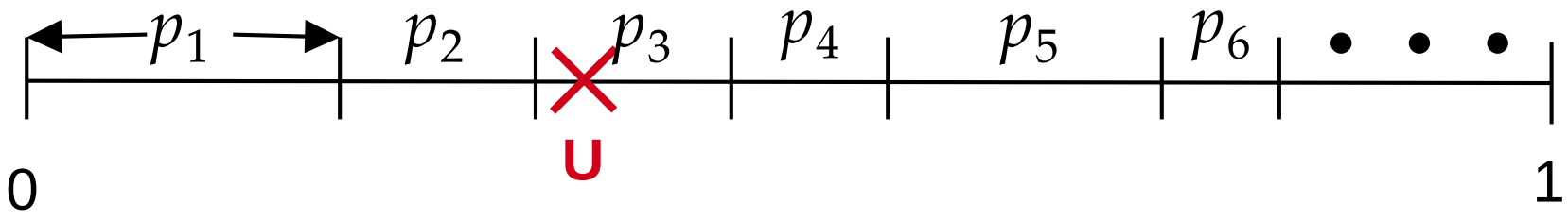
Linear and Binary Search Sampling

Linear search Sampling

Suppose that X has distribution p which takes values in $\{x_1, x_2, \ldots, x_n, \ldots\}$ and that we **know** the values $p_j = \mathbb{P}(X = x_j)$.

Linear search Sampling

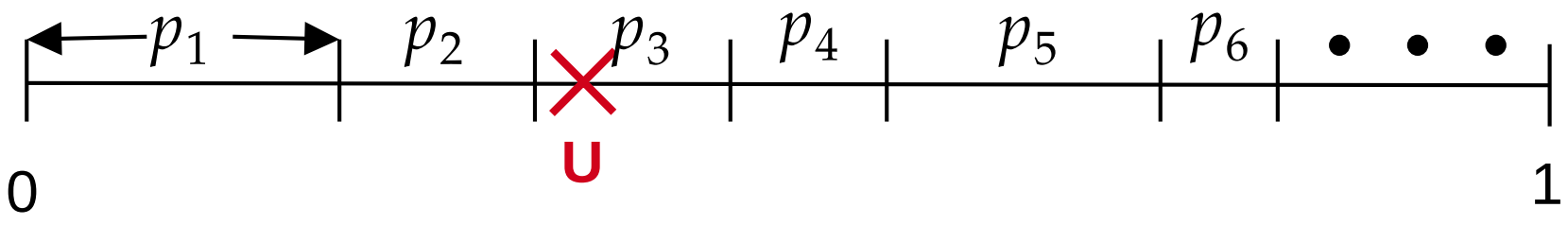
1. Sample U as a random variable with uniform distribution in $[0, 1]$
2. Define $c_0 = 0$
3. Let $U \sim \mathcal{U}(0, 1)$
4. Set $i = 1$
5. Set $c_i = c_{i-1} + p_i$
6. If $U < c_i$, return x_i; otherwise set $i = i + 1$ and go to 5.



Exercise: Show that Linear Sampling actually returns a value X with distribution p as described.

Binary search Sampling

The main idea used to sample from a discrete distribution is given in this picture



Hence, given $U \sim \mathcal{U}(0,1)$, we just need to find *where U* lands.

Linear Sampling looks the intervals from left to right. We can do better by using a **binary search**.

2. Simulation of Discrete Random Variables

Binary Search Sampling

Binary Search Sampling

We know have to compute the CDF.

$$CDF(i) = \sum_{j=1}^{i} p_j$$

This is stored in a vector with n values. The algorithm is

Binary Search Sampling

1. set U to be a uniform in $[0, 1]$
2. Set $low \leftarrow 1$
3. Set $high \leftarrow n$
4. **While** $low \leq high$
   - 4.1 Set $mid \leftarrow \left\lfloor \frac{low+high}{2} \right\rfloor$
   - 4.2 **If** $U \leq CDF[mid]$ **then**
     - 4.2.1 Set $high \leftarrow mid - 1$
   - 4.3 **Else**
     - 4.3.1 Set $low \leftarrow mid + 1$
5. Return low

2. Simulation of Discrete Random Variables

Sampling of common Discrete Random Variables

Sampling discrete random variables

Most programming language have libraries with sampling algorithms for the most common random variables.

Sampling in Python

1. Binomial Distribution: `np.random.binomial(n,p,m)` returns a vector of m independent $\mathrm{Bin}(n, p)$
2. Poisson Distribution: `np.random.poisson(l,m)` returns a vector of m independent $\mathrm{Poi}(l)$
3. Geometric Distribution: `np.random.geometric(p,m)` returns a vector of m independent $\mathrm{Geo}(p)$

In general `np.random` is the generator of everything

Sampling discrete random variables

We can also give a list of values and probabilities:

```
values = np.array([-1, 0, 1, 4])
prob_vector = np.array([0.1, 0.2, 0.3, 0.4])
samples = np.random.choice(values, size=m, replace = True, p=prob_vector)
```

2. Simulation of Discrete Random Variables

Sampling of Combinatorial Objects: Permutations and Random subsets

Random Subsets - Sampling without replacement

Sampling a subject (also known as sampling without replacement) is an important subrutine in simulation of discrete events.

Problem

Given a finite set A and a non-negative integer $k \leq |A|$. How can we sample a random (uniform) k-subset of A.

Recall that there are $\binom{|A|}{k}$ subsets of k elements, so every subset has probability $\binom{|A|}{k}^{-1}$ of being chosen.

Random Subsets - Sampling without replacement

Algorithm for Random Subset

Input: a set A, and an positive integer $k \leq |A|$
Output: a random k-subset of A, chosen uniformly at random

1. $X \leftarrow \emptyset$
2. For $i = 1$ to K
   - 2.1 Choose $x \in A$ uniformly at random
   - 2.2 $X \leftarrow X \cup \{x\}$
   - 2.3 $A \leftarrow A \setminus \{x\}$
3. Return X

Sampling without replacement can be easily implemented in Python by using `np.random.choice`.

```
my_set = np.array([-1, 0, 1, 4,9,10,13,41,-4,12])
k = 5
X = np.random.choice(my_set, size=k, replace=False)
```

Random Permutation

Random permutations are another fundamental primitive for several statistical applications, in particular for bootstrapping methods.

A permutation of the set $A = \{1, \ldots, n\}$ is any ordering of the elements of A.

We can easily note that the algorithm for random subsets give us the elements in a random order!

Hence we just need to use sample a n-subset of $\{1, \ldots, n\}$ using our previous algorithm and that's it.

In Python we can do it with `np.random.permutation` or with choice

```
X = np.random.permutation(n)

A = np.arange(1, n + 1)
Y = np.random.choice(A, size=n, replace=False)
```

np.random.choice

A word of caution.

- `np.random.choice` is known to be slow for large vectors/sets.
- It is a well-known thing. Similar with the function `sample` in `R`.
- In some cases we would need to be a bit hackier to make things faster.
- For example, `Y = np.random.choice(A, size=4, replace=False)` takes a while for A of size 100,000,000, but we can improve its speed.

```python
import numpy as np
import time

n = 100000000
A = np.arange(1, n + 1)
start_x = time.time()
X= np.random.choice(A, size=4, replace=False)
end_x = time.time()
time_x = end_x - start_x

print(time_x)
print(X)

start_y = time.time()
not_ready = True
while(not_ready):
    Y = np.random.randint(1,n+1,size = 4)
    if np.unique(Y).size == Y.size:
        not_ready = False

end_y = time.time()
time_y = end_y - start_y

print(time_y)
print(Y)
```

3. Simulation of Continuous Random Variables

We will study some techniques to simulate continuous random variables. In particular:

1. Inverse Transformation Method
2. Methods to generate normal random variables

3. Simulation of Continuous Random Variables

Inverse Transformation Method

Sampling from continuous distributions on $\mathbb{R}$ is very hard in general, and knowing the density function is not enough.

It is important to compare with discrete distributions where knowing the probability mass function is enough to derive an algorithm for virtually any distribution (despite the fact that they will be very slow, will have memory issues, and numerical issues as well...)

But, if we know the cumulative distribution function F of a continuous random variable, then we can sample from it.

Proposition

Let X be a continuous random variable taking values in $\mathbb{R}$ with cumulative distribution function F. Then, the random variable $U = F(X)$ has uniform distribution in $[0, 1]$.

Proposition

Let X be a continuous random variable taking values in $\mathbb{R}$ with cumulative distribution function F. Then, the random variable $U = F(X)$ has uniform distribution in $[0, 1]$.

By using the proposition we derive the following algorithm:

Inverse Sampling Algorithm

1. Sample $U \sim \mathcal{U}(0, 1)$
2. Find X such that $U = F(X)$
3. return X

When F is strictly increasing we may be able to calculate F^{-1} explicitly. In such a case we can just return the value $X = F^{-1}(U)$. Otherwise, we may need to solve the equation $U = F(X)$ for a value U. There are many methods for solving equations, but we will not study them here.

Exercise: Show that if $U \sim \mathcal{U}$, then $-\lambda \log(U) \sim \text{Exp}(\lambda)$

3. Simulation of Continuous Random Variables

Generating Normal Random Variables

Generating from normal random variables is not easy. There are a couple of ideas that can be considered. Most of them have a geometric flavour.

Algorithm 1 Box-Muller transform

Ensure: Two independent samples of $\mathcal{N}(0, 1)$

Sample $U_1, U_2 \overset{i.i.d.}{\sim} \mathcal{U}$;
$Z_1 = \sqrt{-2\log(U_1)}\cos(2\pi U_2)$;
$Z_2 = \sqrt{-2\log(U_1)}\sin(2\pi U_2)$;
return (Z_1, Z_2)

In Python, we generate n independent normals by using `np.random.normal(mu, sigma, n)`

Simulating multivariate distributions is relatively easy by using the fact that all multivariate normal random variables are an affine transformation of a vector of independent standard normals.

Algorithm 2

Require: $\mu \in \mathbb{R}^d$ and $\Sigma \in \mathbb{R}^{d \times d}$ definite positive
Ensure: $X \sim \mathcal{N}(\mu, \Sigma)$

Sample $(Z_i)_{i=1}^d \overset{i.i.d}{\sim} \mathcal{N}(0, 1)$;
$Z \leftarrow (Z_1, \ldots, Z_d)$
Compute A such that $AA^\intercal = \Sigma$
$X \leftarrow \mu + AZ$
return X

The difficult step of the previous algorithm is to compute A, however, there are many linear algebra algorithms to do this, e.g. Cholesky's decomposition. We **may** have a look at this later in the course.

In Python, we generate n independent multivariate normals by using

```
np.random.normal(mean, Sigma, n)
```

4. A bit of Reproducibility

A bit of reproducibility

In general, we want to be able to repeat our experiment, even if they are random.

If we want reproducibility, instead of using `np.random` to generate random variables, we should create a generator with a fixed seed

```
rng = np.random.default_rng(2025)
X = rng.poisson(1,100)
print(X)
```

In the previous code, `rng` is the object that will be in charge of generating random variables with the seed 2025.

This is much better as we can have different random generators (e.g. one for simulate data, one for random algorithms, etc)

5. Sampling from unnormalised Distributions

Sampling from unnormalised distributions

We want to sample from a probability mass function p that put probability in a discrete set S.

The problem is that we know p up to a constant. That is, instead of knowing p, we know q such that

$$p(x) = q(x)/Z$$

Of course, if we can compute Z and the set of possible values of x is not too large, then we can use the standard methods to sample from p.

In particular, we would be able to use `np.random.choice`

5. Sampling from unnormalised Distributions

Motivation: Boltzmann distributions

Boltzmann distributions

For example, consider $\mathcal{H}_n = \{(b_1, \ldots, b_n) : b_i \in \{0, 1\}\}$. H_n has 2^n elements. A nice way to construct interesting stuff is to consider a *Hamiltonian*, which is just a function that measures some sort of *energy*.

For example:

- $H(b_1, \ldots, b_n) = -\sum_{i=1}^{n} b_i$
- $H(b_1, \ldots, b_n) = -\sum_{i=1}^{n} \sum_{j=1}^{n} ij \mathbb{1}_{\{b_i = b_j\}}$
- $H(b_1, \ldots, b_n) = \left(\sum_{i=1}^{n} b_i\right)^2$
- $H(b_1, \ldots, b_n) = \sum_{i=1}^{n} \sum_{j=1}^{n} \frac{1}{i+j} b_i b_j$

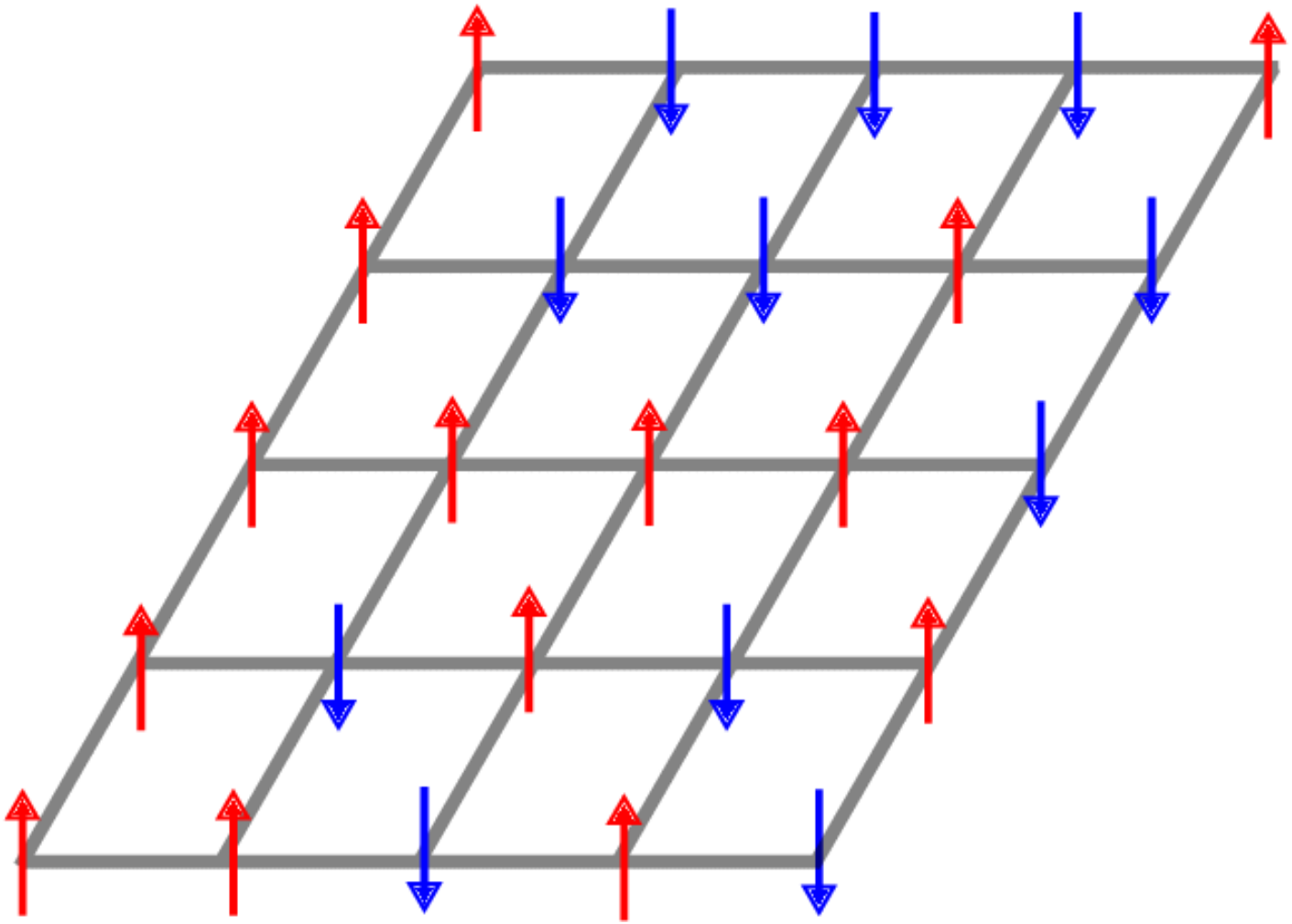
Then, we want to sample from the pmf

$$f(b) = \exp\left(\beta H(b)\right) / Z(\beta)$$

here

- $b = (b_1, \ldots, b_n) \in \mathcal{H}_n$
- β is a real-valued parameter (inverse of absolute temperature, usually $\beta > 0$ for correct interpretation)
- $Z(\beta)$ is the partition function so $\sum_{b \in \mathcal{H}_b} f_\beta(b) = 1$.

Boltzmann distributions



Boltzmann distributions

Finding Z hard in general. In some cases, we can obtain closed-form formulas that can be computed relatively quickly, in some other cases it is really hard.

Example 1: In a simple case the Hamiltonian $H(b_1, \ldots, b_n) = -\sum_{i=1}^{n} b_i$ gives a model with n independent coins (bits), and the probability of a head ($b_i = 1$) is $e^{\beta}/(1 + e^{\beta})$. We can easily simulate from this.

Example 2: If we consider $H(b_1, \ldots, b_n) = \left(\sum_{i=1}^n b_i\right)^2$, then $Z(\beta) = \sum_{k=0}^n \binom{n}{k} e^{\beta k^2}$. We can simulate from the pmf

$$f(b) = \frac{e^{\beta\left(\sum_{i=1}^n b_i\right)^2}}{\sum_{k=0}^n \binom{n}{k} e^{\beta k^2}}$$

but we need to think!

1. Notice that all seuqneces of bits with the same number of 1's have the same probability
2. What is the probability of sampling a sequence of j-bits?
3. $p(j) = \frac{\binom{n}{j} e^{\beta j^2}}{\sum_{k=0}^n \binom{n}{k} e^{\beta k^2}}$
4. We can sample from $p(j)$, $j \in \{0, \ldots, n\}$ by using `np.random.choice`. At least for n not so large! (why?)
5. Once we sample from $J \sim p$, we just need to distribute the J values 1's into the n bits, and all the rest are 0. This is equivalent to choose J random elements of a set of n, without resampling. This can be done with `np.random.choice`.

```python
import numpy as np
from scipy.special import comb

def sample_bitstring(n, beta):
        #Step 1: compute p(j) and Z
    p = np.zeros(n + 1)
    Z = 0
    for j in range(n + 1):
        p[j] = comb(n, j) * np.exp(beta * j**2)
        Z = Z+p[j]
    p = p/Z

    # Step 2: Sample from p
    J = np.random.choice(np.arange(n + 1), p=p)
    # Step 3: distribute the J ones
    bitstring = np.zeros(n)
    ones_indices = np.random.choice(np.arange(n), size=J, replace=False)
    bitstring[ones_indices] = 1

    return bitstring

n = 50
beta = 1.0

bitstring = sample_bitstring(n, beta)
print(bitstring)
```

Despite our efforts, our algorithm has several problem. The main problems are that

- `np.exp(710)` is `inf`, so from that point all the computations are nonsense
- `comb(1100,550)` is `inf`, so more nonsense

So our algorithm works for n up to 709, but probably less due to numerical errors

Boltzmann distributions

In general, a very hot research topic is the understanding of Boltzmann distributions, either in theory and practice.

A particular example of interest is

$$H(b) = \sum_{i=1}^{n} \sum_{j=1}^{n} A_{ij} b_i b_j$$

where A is the adyacency matrix of a graph.

- if $b_i \in \{+1, -1\}$ then, the associated distribution yields de so-called Ising model or Hopfield networks
- If $b_i \in \{0, 1\}$ then the model is called Boltzmann Machine

Boltzmann distributions

We can also consider other spaces for Boltzmann distributions, instead of H_n.

In the real line we can consider the Hamiltonian $H(x) = -x^2/2$, so we get the density function

$$f(x) \propto \exp(\beta H(x))$$

which is no more than a normal with mean 0 and variance $\sigma^2 = 1/\beta$.

Nevertheless, we can go further and introduce very complex distributions

5. Sampling from unnormalised Distributions

Rejection Method

The Rejection Method

We introduce the **rejection method** in the discrete setting.
For rejection sampling we consider the following:

1. A probability mass function g on the points $\{x_1, x_2, \ldots\}$.
2. A function $h : \{x_1, x_2, \ldots\} \to [0, 1]$ such that $\sum_{i=1}^{\infty} g(x_i)h(x_i) > 0$.

Abstract Rejection Sampling

1. Sample X according to g
2. With probability $h(X)$, return X; otherwise go back to step 1.

Proposition

The probability mass function of X is proportional to the function $q(x) = h(x)g(x)$.

Rejection Sampling Algorithm

Suppose we are given a function $q(x)$, and we want to sample from $p(x) = q(x)/Z$.

We proceed as following:

1. Find a pmf g such that for all x, $q(x)/g(x) \leq M < \infty$ for some constant M.
2. We assume here that we can sample from g.
3. Choose $h(x) = q(x)/(Mg(x))$
4. Use the abstract Rejection Sampling with g and h above.

Remark 1: The pmf g is called the envelope

Rejection Sampling Algorithm

Rejection Sampling Algorithm

Input: g such that $q/g \leq M$

1. Sample X according to g
2. Generate a random variable U
3. if $U \leq q(X)/(Mg(X))$, return X; otherwise go to 1.

Proposition

The Rejection Sampling generates samples X with pmf f, and the number of samples of Y (step 1) needed by the algorithm is a geometric random variable with mean M.

The Rejection Method

Example:

- Consider the a pmf f with mass on $\{-3, -2, -1, 0, 1, 2, 3, 4, 5, 6\}$ with corresponding probability vector proportional to

$$q = (8, 9, 13, 9, 12, 12, 1, 11, 11, 14).$$

- We can consider g uniform on $\{-3, -2, -1, 0, 1, 2, 3, 4, 5, 6\}$ with mass $1/10$ on each of those points. We can easily sample from g by using `np.random.randint`
- Finally, $max_x q(x)/g(x) = \frac{14}{0.1} = 140$, so we set $M = 140$

Example Cont.

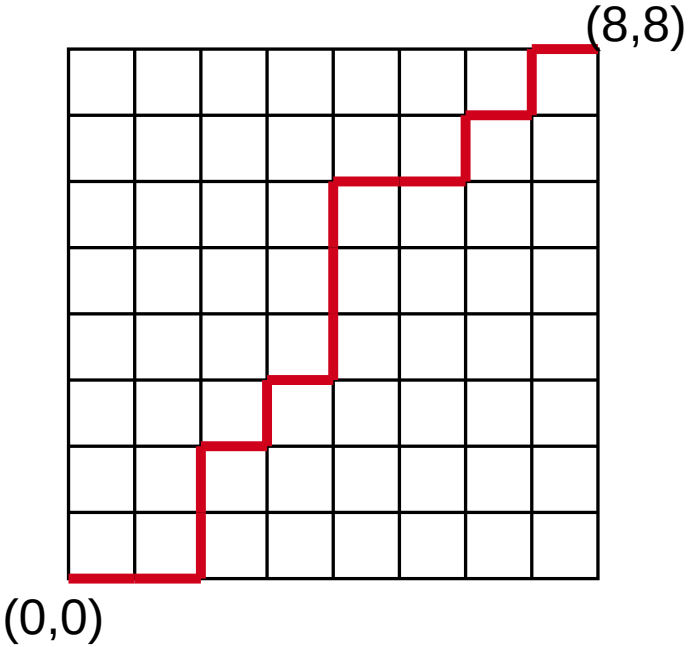
```
import numpy as np

q = np.array([8, 9, 13, 9, 12, 12, 1, 11, 11, 14])
l = len(q)
M = max(q) * l

not_ready = True
while not_ready:
    X = np.random.randint(0, l)
    U = np.random.uniform()
    if U < (l * q[X] / M):
        X = X - 3
        not_ready = False

print(X)
```

A hard problem: Sampling paths

Problem

Given a $n \times n$ grid. Let $\mathcal{P}$ be the set of all path that go from the bottom-left corner to the top-right corner only using $\uparrow$ and $\rightarrow$ steps. To each path P let $c(P)$ be the number of corners. We introduce the probability measure

$$p(P) = \frac{c(P)^{-k}}{\sum_{P' \in \mathcal{P}} c(P')^{-k}}$$



A hard problem: Sampling paths

How can we sample from the distribution

$$p(P) = \frac{c(P)^{-k}}{\sum_{P' \in \mathcal{P}} c(P')^{-k}}$$

Enumerating is a bad idea because there are $\binom{2n}{n} \approx \frac{4^n}{\sqrt{\pi n}}$ paths.
It would take forever to enumerate all the paths.

A hard problem: Sampling paths

Let's try using rejection sampling. For that we need to be able to sample paths easily.

Sampling uniformly is easy: **why?**

In the uniform distribution each path has probability $g(P) = \binom{2n}{n}^{-1}$

A hard problem: Sampling paths

Let's apply the recipe for Rejection Sampling.

1. set $q(P) = c(P)^{-k}$
2. set $g(P) = \binom{2n}{n}^{-1}$
3. We need to find M such that $q(P)/g(P) \leq M$ for all paths P

$$q(P)/g(P) = \frac{\binom{2n}{n}}{c(P)^k} \leq \binom{2n}{n} =: M$$

4. then we choose $h(P) = q(P)/(Mg(P))$

$$h(P) = \frac{\binom{2n}{n}}{c(P)^k \binom{2n}{n}} = \frac{1}{c(P)^k}$$

5. Use the abstract Rejection Sampling with g and h.

A hard problem: Sampling paths - The final algorithm

```
import numpy as np

#A path is represented by a bunch of 0 and 1.
#0 means right, 1 means up
#each path has n values 0, and n values 1.

def sample_uniform_path(n):
    path = np.zeros(2*n)
    path[np.random.choice(np.arange(0,2*n),n, replace = False)]=1
    return path

#probablemente hacer un ciclo for para contar esquinas es mejor que esta solución
def count_corners(arr):
    arr = np.array(arr)
    shifted_arr = np.roll(arr, -1)
    shifted_arr[len(arr)-1]=arr[len(arr)-1]
    pattern_changes = (arr != shifted_arr)
    return np.sum(pattern_changes)

def rejection_sampling(n,k=1):
    ready = False
    while not ready:
        path = sample_uniform_path(n)
        corners = count_corners(path)

        exp = np.random.exponential(1,1)
        if exp>k*np.log(corners):
            ready = True
    return(path)
```

The algorithm is reasonable for $k = 1$, but it takes forever for $k = 2$.

Why?

An heuristic

The probability of accepting a path is $1/c(P)^k$.

- If we get a path with very few corners we are likely going to accept it, e.g. $c(P) = 1$ or $c(P) = 2$
- But on a path of length $2n$, a random path (uniform) has on average n corners, and standard deviation of order $\sqrt{n}$. So **MOST** of the sampled path have around n corners. The probability of accept one of them is $\frac{1}{n^k}$, i.e. very unlikely.
- In other words, the algorithm is waiting for us to propose paths with less corners (since it is easier to accept them)
- but random path have several corners
- We need to find a distribution g that makes $h(p)$ as large as possible, but at the same time we need to be able to simulate from it.

The Rejection Method

VERY IMPORTANT: The Rejection Method can be applied for continuous random variables by replacing the pmf by densities.

This technique is actually more useful in the continuous setting where less sampling techniques are available.

Rejection Method for Continuous Random Variables

The rejection method for continuous random variables works the same as with discrete random variables, but using densities instead of probability mass functions.

Recall that we want to simulate from a density f by using a density g

Rejection Sampling Algorithm

Input: g such that $f/g \leq M$

1. Sample Y according to g
2. Generate a random variable U
3. if $U \leq f(Y)/(Mg(Y))$, set $X = Y$ and return X; otherwise go to 1.

5. Sampling from unnormalised Distributions

Metropolis-Hasting

Metropolis-Hasting

Another way of sampling of complicated distributions is by using Metropolis-Hasting.

The theory here is complicated, so we will focus mostly in the algorithm

This is a special case of a MCMC algorithm, which we **may** study in detail later in the course

Metropolis-Hasting

In most MCMC algorithm we want to sample from a distribution p which can be known up-to-constants.

The idea: start with a candidate and start modifying it little by little until a good sample is generated.

Rejection Sampling is an example of an algorithm like the one above, but the new candidates are completely uncorrelated with the previous one.

This is bad if the candidates we generate have a large chance of being bad ones, and this is good if we are generating good candidates (e.g. from a distribution that is close to the target).

However, in most cases we are in the bad case, so we need to improve the candidate little by little.

Metropolis-Hasting

The Metropolis-Hasting algorithm requires the following elements

1. A function $q(x)$ such that $p(x) = q(x)/Z$
2. A proposal distribution $Q(x, x')$ which tells me how to generate a new candidate X' given a current candidate X.

Metropolis-Hasting: running example

As an example, let's go back to our problem of sampling from a random element of $\mathcal{H}_n = \{(b_1, b_2, \ldots, b_n) : b_i \in \{0, 1\}\}$ according to the probability distribution

$$p(x) \propto e^{\beta\left(\sum_{i=1}^{n} b_i\right)^2}$$

Here, a candidate is a vector of n bits.

A new candidate can be, for example, choosing a random bit and change modify its value.

Mathematically

$$Q(x, x') = \frac{1}{n}\mathbb{1}\{x \text{ and } x' \text{ differ in exactly one bit}\}$$

Metropolis-Hasting: running example

The Metropolis-Hasting algorithm is the following:

1. Propose an initial candidate X_0
2. Given a candidate X_t, propose X' from $Q(X_t, \cdot)$
3. With probability $A(X_t, X') = \min\left(1, \frac{P(X')Q(X', X_t)}{P(X_t)Q(X_t, X')}\right)$ make $X_{t+1} = X'$, otherwise make $X_{t+1} = X_t$.
4. Repeat step 2 **a lot of times**

Metropolis-Hasting: running example

In our running example we have

1. $p(x) \propto e^{\beta\left(\sum_{i=1}^{n} b_i\right)^2}$.
2. $Q(x, x') = \frac{1}{n}\mathbb{1}\{x \text{ and } x' \text{ differ in exactly one bit}\}$.
   In simple words, change one bit at random!
3. Note that $Q(x, x') = Q(x', x)$
4. Then, we accept candidates with probability $A(X_t, X) = \min\left(1, \frac{p(X')}{p(X_t)}\right) = \min\left(1, \frac{p(X')}{p(X_t)}\right)$

5. That is, Sample $U \sim \mathcal{U}(0,1)$, then if $U \leq \min\left(1, \frac{p(X')}{p(X_t)}\right)$, then $X_{t+1} = X'$, otherwise $X_{t+1} = X_t$
6. Here is where taking logarithms is key:

$$\log U \leq \min\left(0, \beta\left(\left(\sum_i b_i'\right)^2 - \left(\sum_i b_i^t\right)^2\right)\right)$$

Alternatively

$$-\log U \geq \max\left(0, \beta\left(\left(\sum_i b_i^t\right)^2 - \left(\sum_i b_i'\right)^2\right)\right)$$

Avoiding a lot of numerical issues due to the exponential.

Metropolis-Hasting: running example

In the end, this is the condition we accept a new candidate

$$-\log U \geq \max\left(0, \beta\left(\left(\sum_i b_i^t\right)^2 - \left(\sum_i b_i'\right)^2\right)\right)$$

A few remarks

- $-\log U$ has exponential distribution with mean 1, so just generate the exponential.
- Note that we always accept the transition if $\sum_i b_i^t - \sum_i b_i$ is negative. This happens when we transform one 0 into 1, hence giving preference to configurations with more $1'$s, which makes sense since $p(x) \propto e^{\beta\left(\sum_{i=1}^n b_i\right)^2}$

the final algorithm

```
import numpy as np

def Metropolis_Hasting_for_Gibbs(n,beta):
    X = np.random.binomial(1, 1/2,n)
    M = int (2*np.log(n)*n)
    for i in range(M):
        q = np.random.randint(0,n,1)
        Xp = X.copy() #this is very important when you make a copy of a vector/list
        Xp[q]=1- Xp[q]
        E = np.random.exponential(1,1)
        if E>np.max([0, beta*(np.sum(X)**2 - np.sum(Xp)**2)]):
            X = Xp
    return X


n = 100
beta = 0.005
n_samples = 1000
samples = np.zeros((n_samples,n))

for i in range(n_samples):
    samples[i,:] = Metropolis_Hasting_for_Gibbs(n,beta)
```

Metropolis-Hasting

When we develop a Metropolis-Hasting algorithm, we will face the usual question: When do we stop generating candidates?

In technical words: When do we stop the Markov Chain?

This is a problem of all MCMC algorithm

There are some tests to test that your current candidate is "good enought". You can also use Theory. In both cases is a lot of a case to case basis.

6. Extra Material: Composition and Alias Method

Composition Method and Alias Method

Composition Method

We return to the general problem of sampling from a probability mass function.

Suppose we want have two pmf f_1 and f_2 and we know how to efficiently sample from them.

Then we can easily sample from the pmf

$$f(x) = \alpha f_1(x) + (1 - \alpha) f_2(x),$$

- Sample $X_1 \sim f_1$ with probability α, otherwise sample $X_2 \sim f_2$.
- Return the sampled value

In general, this can be extended to general convex combinations (also known as mixtures)

$$f(x) = \sum_{i=1}^{k} \alpha_i f_i(x)$$

with $\alpha = (\alpha_1, \ldots, \alpha_k)$ is a probability vector, and f_i are pmf.
Remark: this method also apply if we use densities instead of pmf.

Alias method

The Alias method is a very clever application of the composition method. It is a very efficient sampling algorithm for discrete random variables taking values on a finite set. The method is also very interesting and creative
The Alias method try to write the pfm of the random variable as a mixture of easy-to-sample random variables

Theorem

Let p be a pmf such that $|\operatorname{supp}(p)| = n \geq 1$. Then, there exist pmf's $q_1, \ldots, q_n$ such that

$$p(x) = \frac{1}{n} \sum_{i=1}^{n} q_i(x)$$

with $|\operatorname{supp}(q_i)| \leq 2$

Essentially, the theorem states that every discrete distribution with finite support can be seen as a uniform mixture of pmf with support of size at most 2.

Idea: If we find such a decomposition, sampling from X will be super easy. **Why?**

Alias method

Example

Consider $p = (1/3, 1/2, 1/6)$.

Then, choose

$$\begin{aligned} q_1 &= (1, 0, 0) \\ q_2 &= (0, 1, 0) \\ q_3 &= (0, 1/2, 1/2) \end{aligned}$$

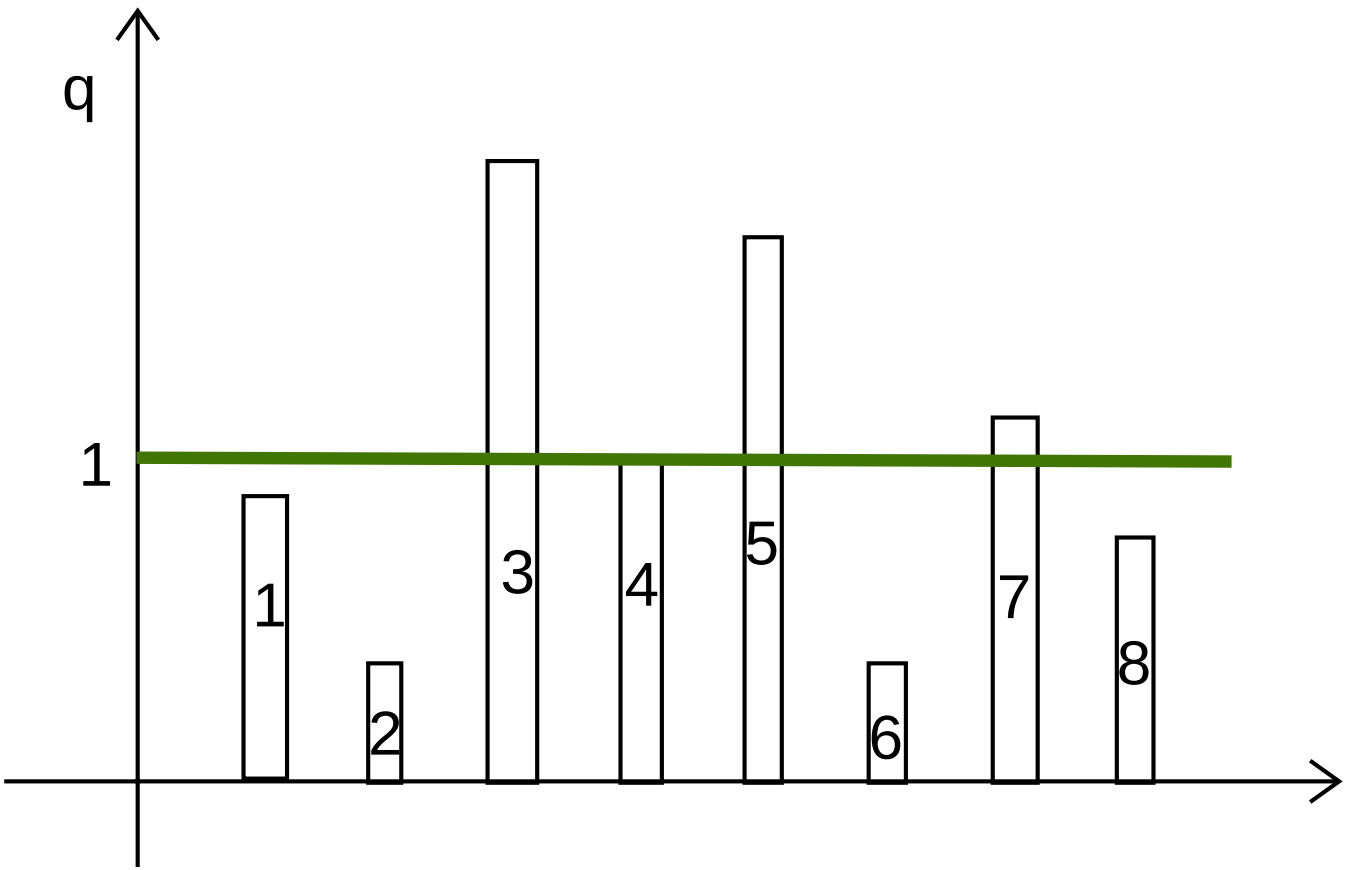
Thus $p = \frac{1}{3}(q_1 + q_2 + q_3)$

Alias method - Walker's Algorithm

To use the theory in our favour, we need to be able to construct the pmf q_i. Walker (1974, 1977) proposed a very ingenious algorithm to do this.

1. Start by multiplying the probability vector p by its length n, so $\sum_{i=1}^{n} np_i = n$. Denote $W = np$
2. We can think about the vector q as buckets of water.
3. To each bucket we will associate another empty box, the **alias**
4. We can separate the buckets into three groups depending if the level of water is **A**bove 1, **B**elow 1, or exactly 1 (**E**xactly **F**ull)

Alias method - Walker's Algorithm



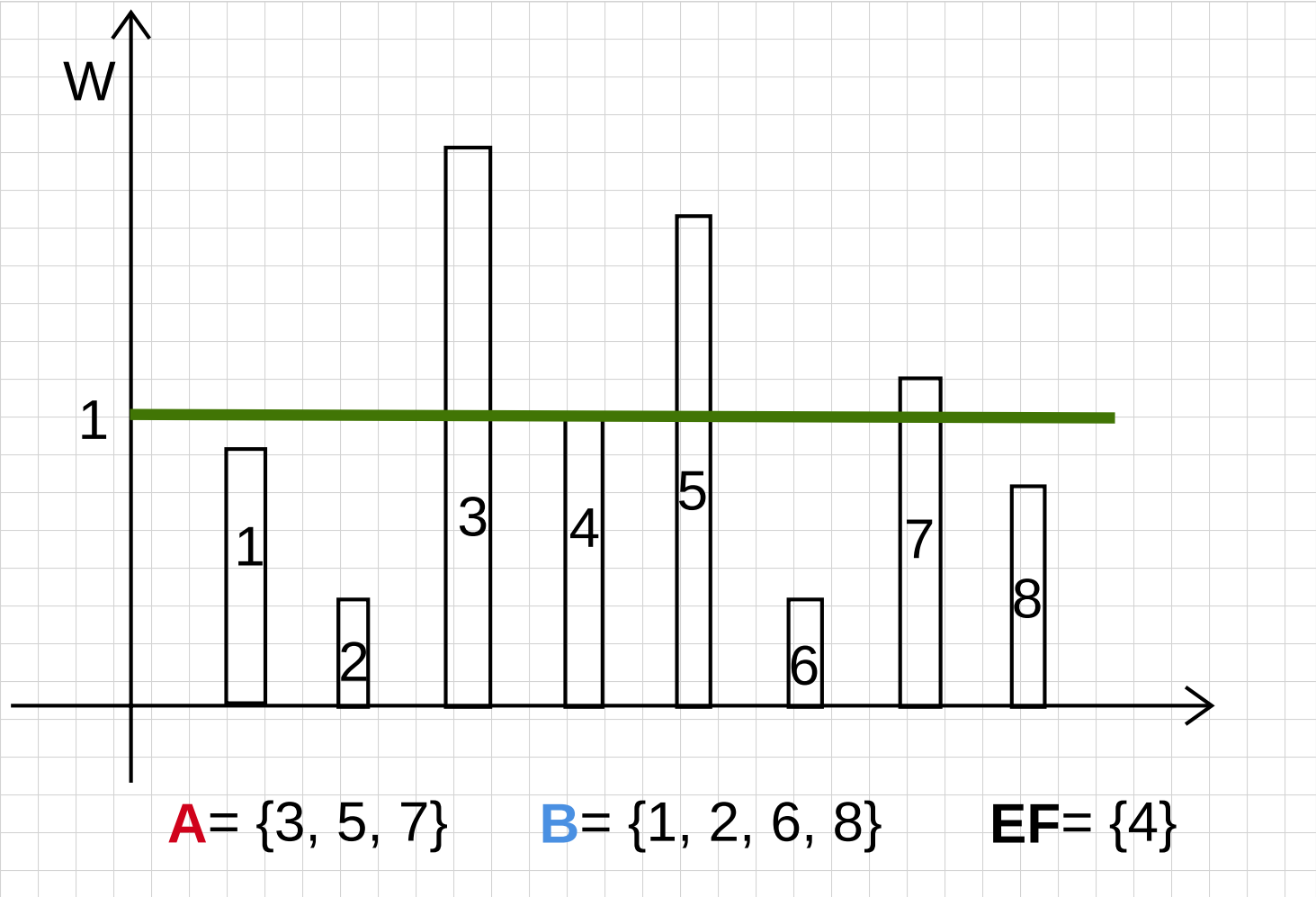
A= {3, 5, 7} **B**= {1, 2, 6, 8} **EF**= {4}

Alias method - Walker's Algorithm

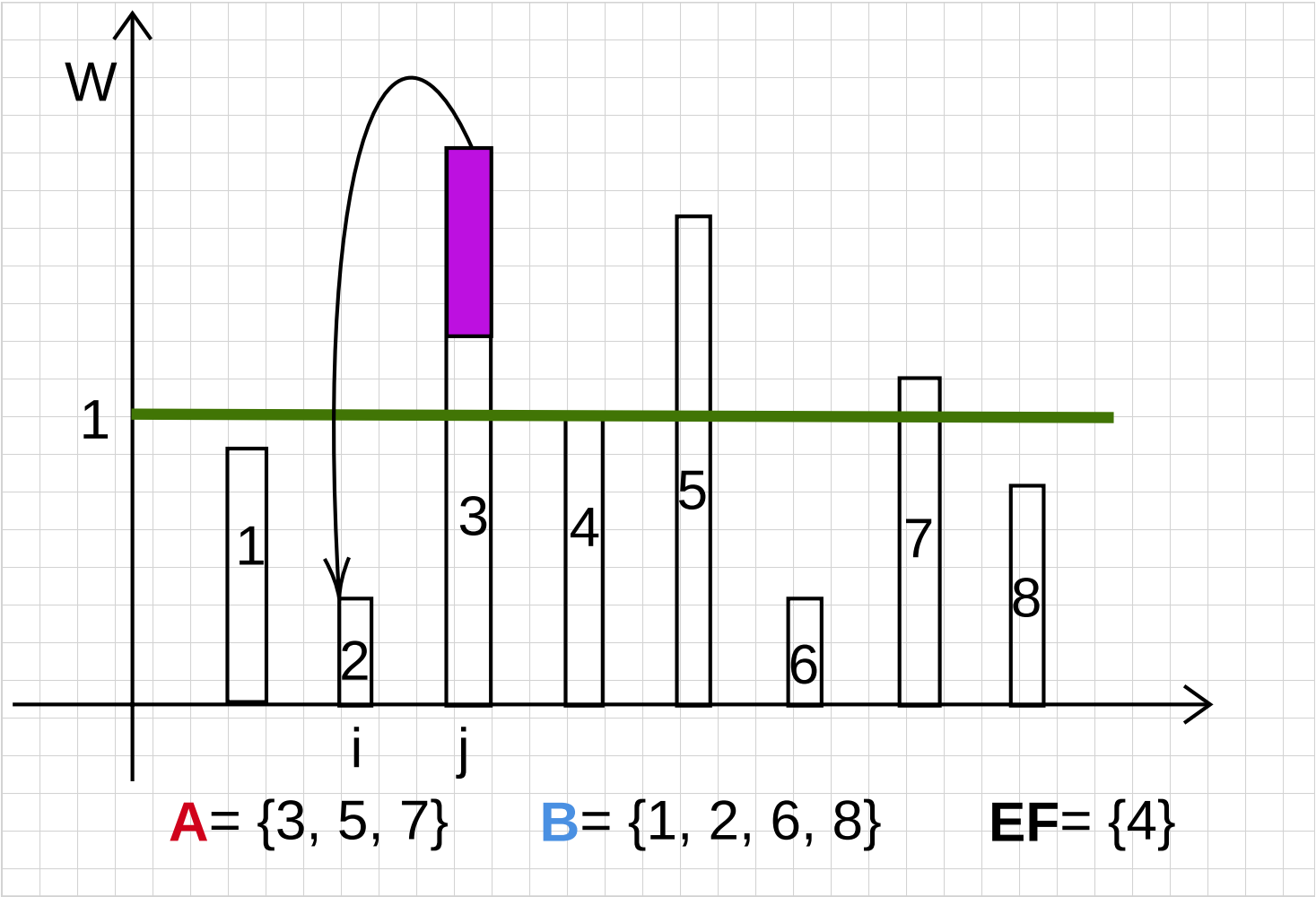
As long as $|B| > 0$ we do the following.

- Choose an arbitrary index $i \in B$ and an arbitrary index $j \in A$.
- Set the alias of i to be j, and move enough water from j to i to make i exactly full.
- Move i to **EF**, and if needed, move j to B.

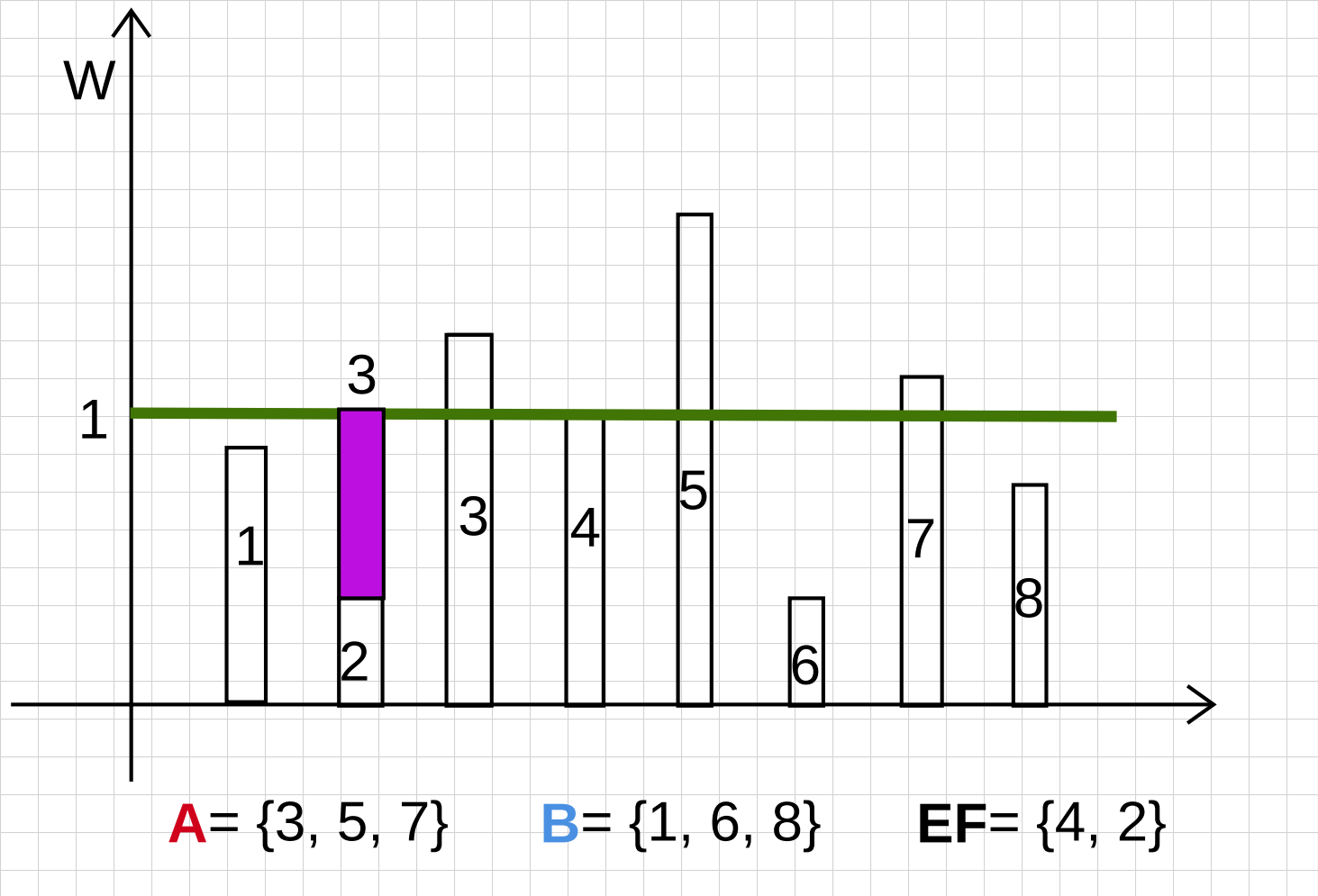
Alias method - Walker's Algorithm



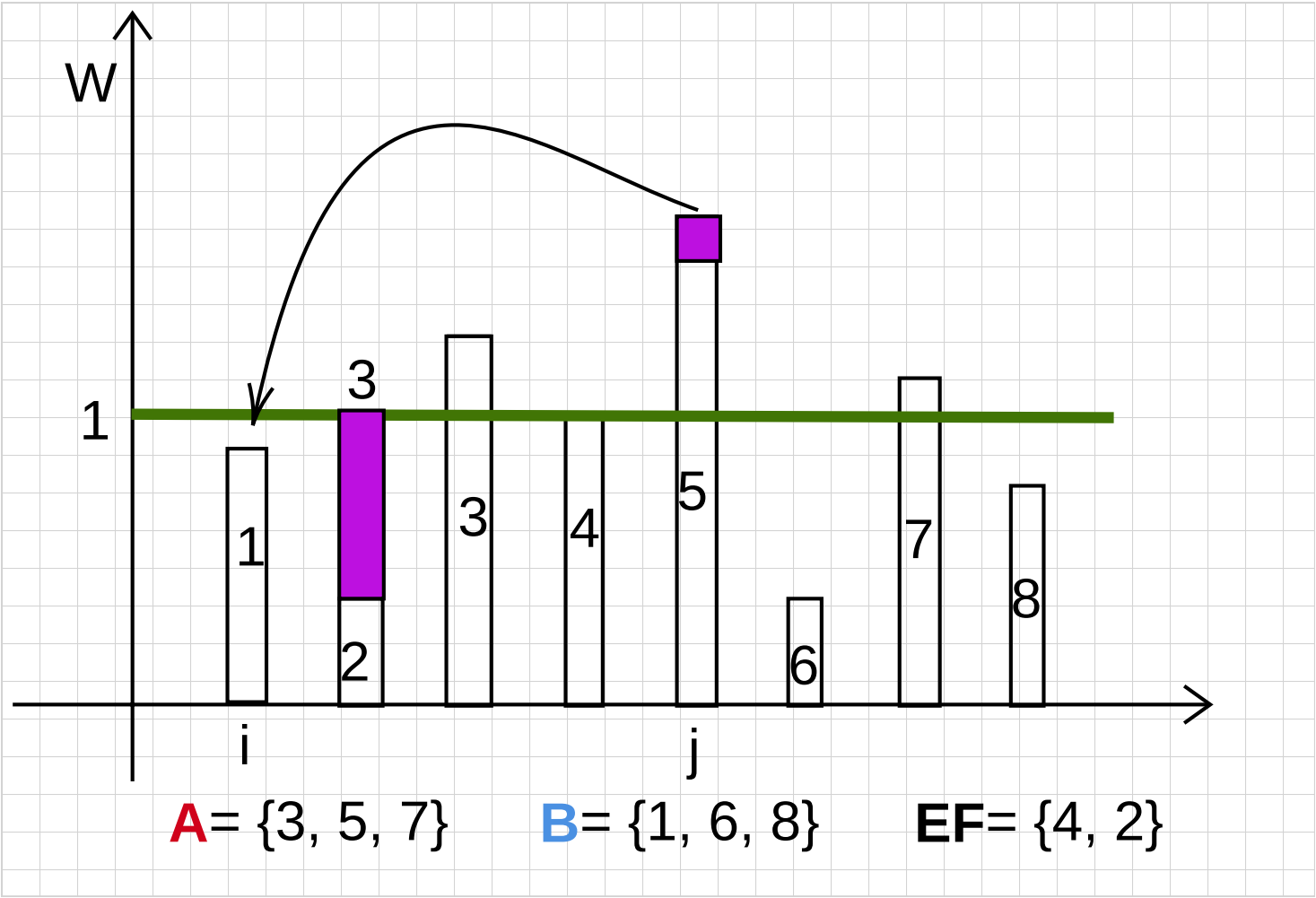
Alias method - Walker's Algorithm



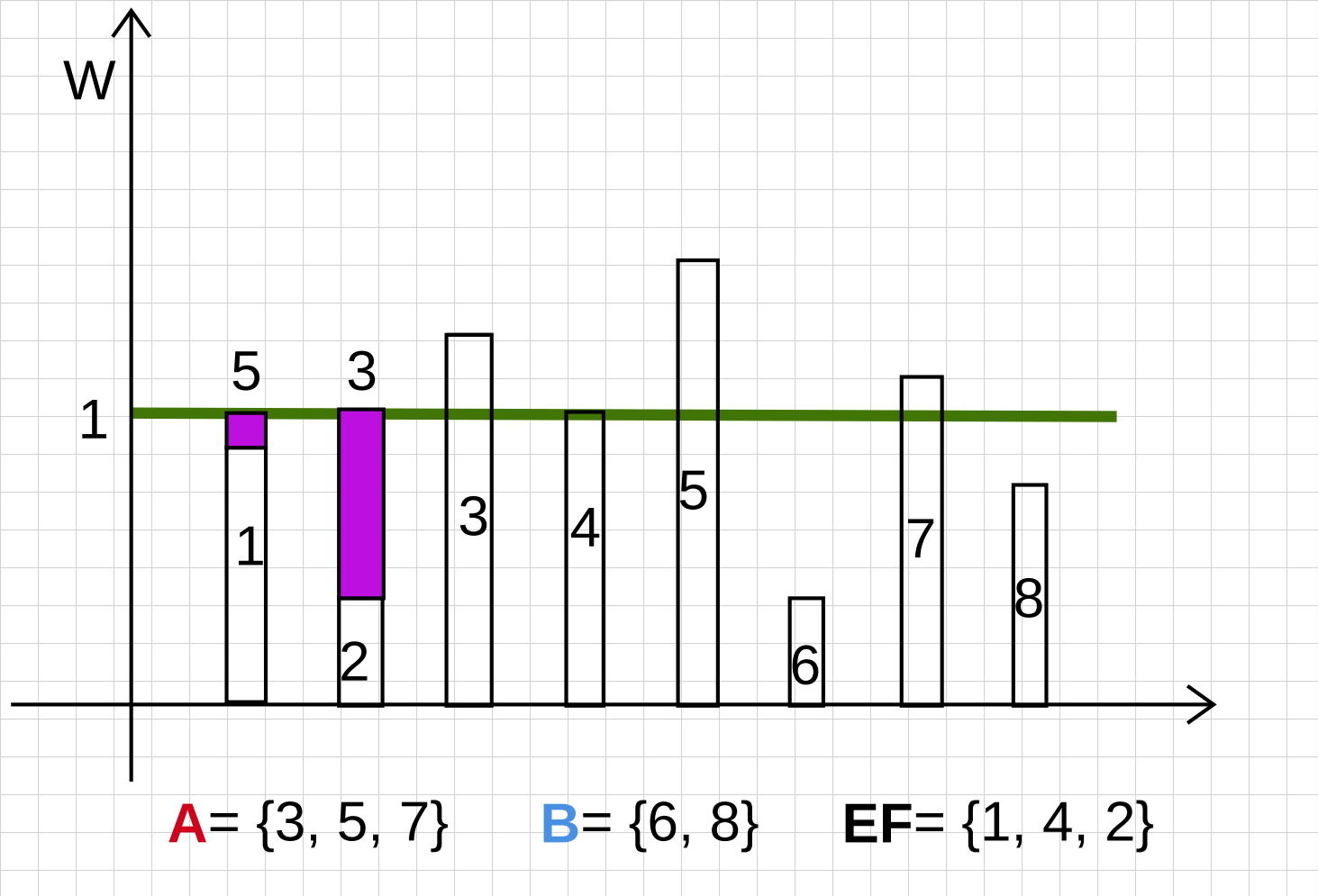
Alias method - Walker's Algorithm



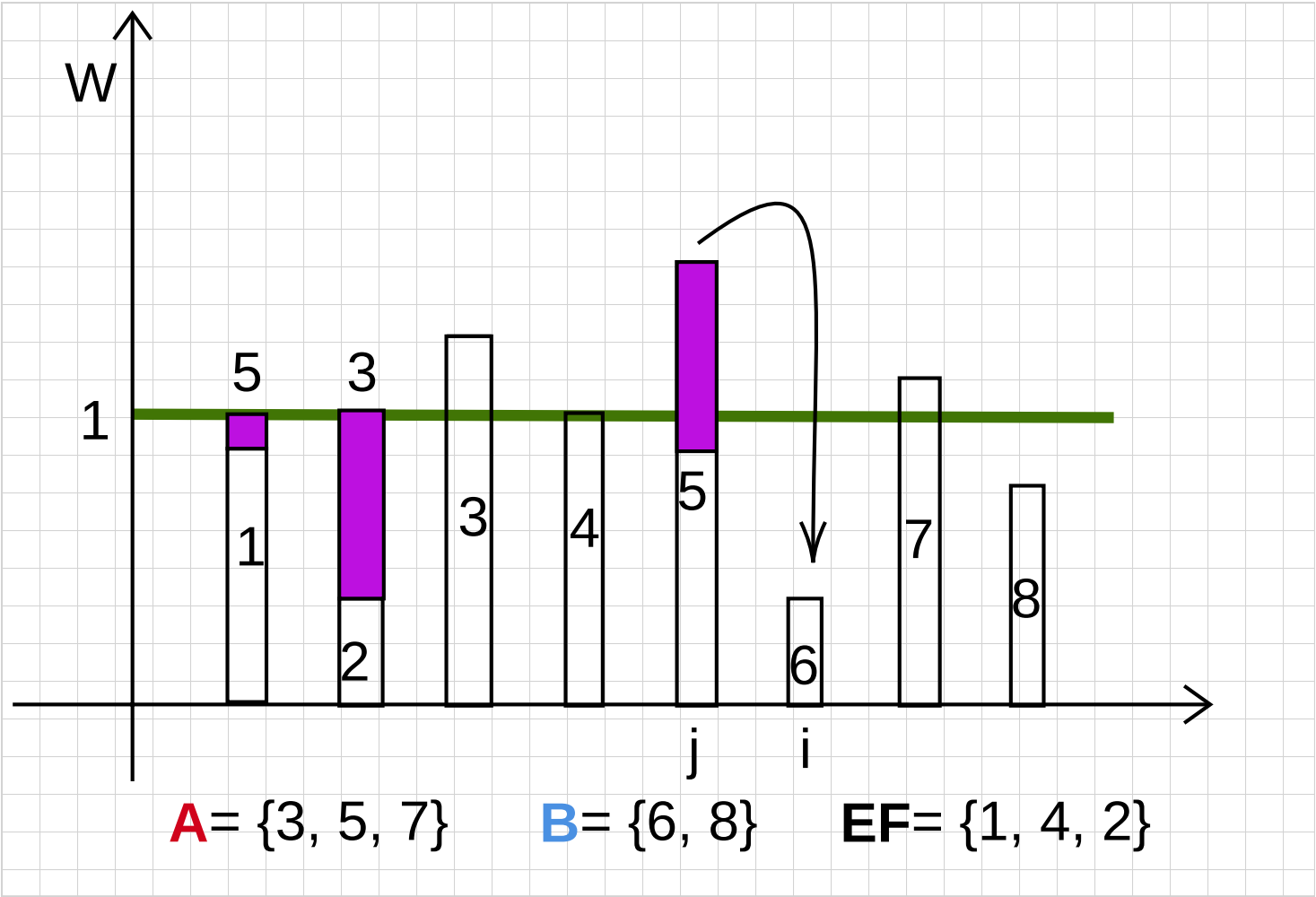
Alias method - Walker's Algorithm



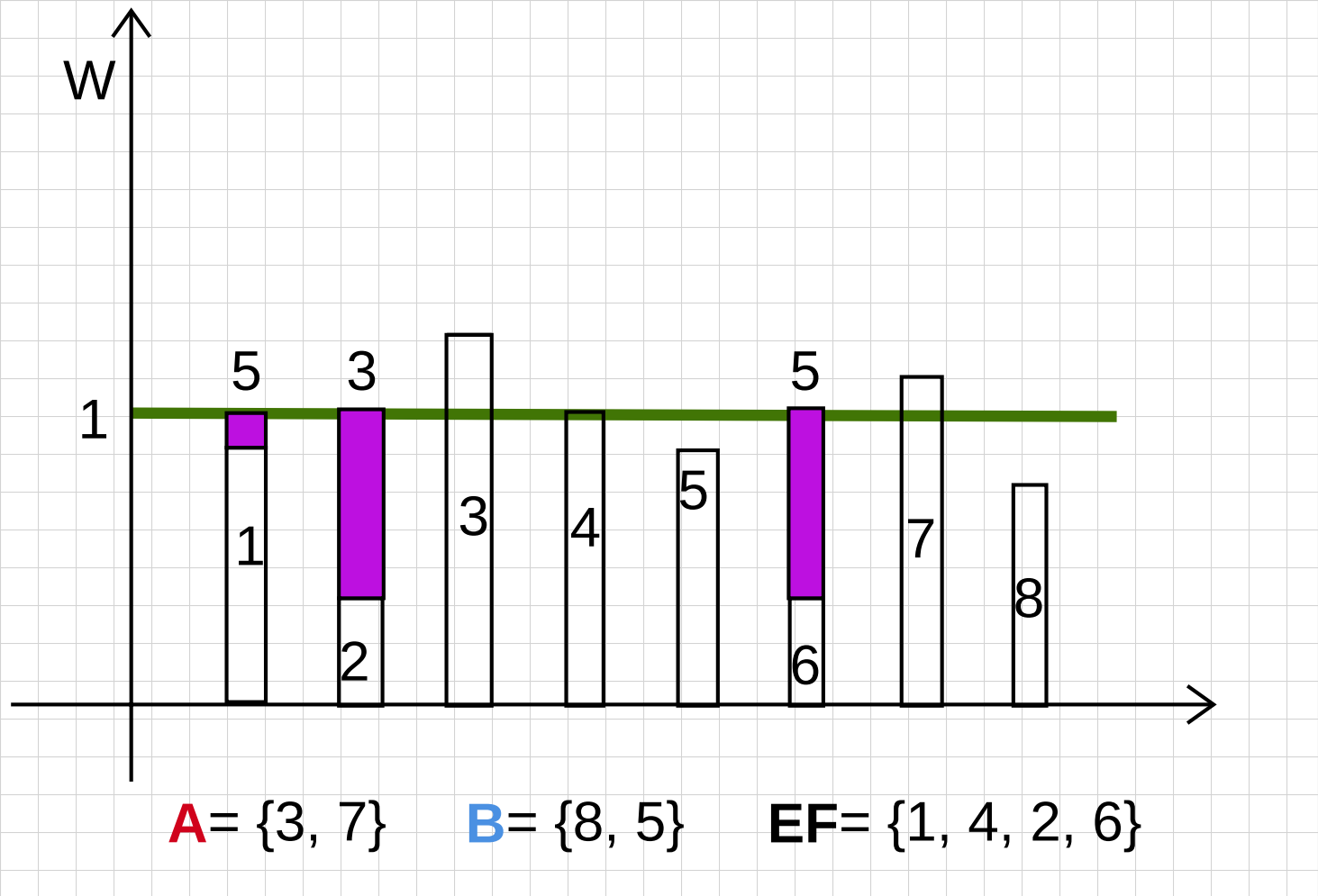
Alias method - Walker's Algorithm



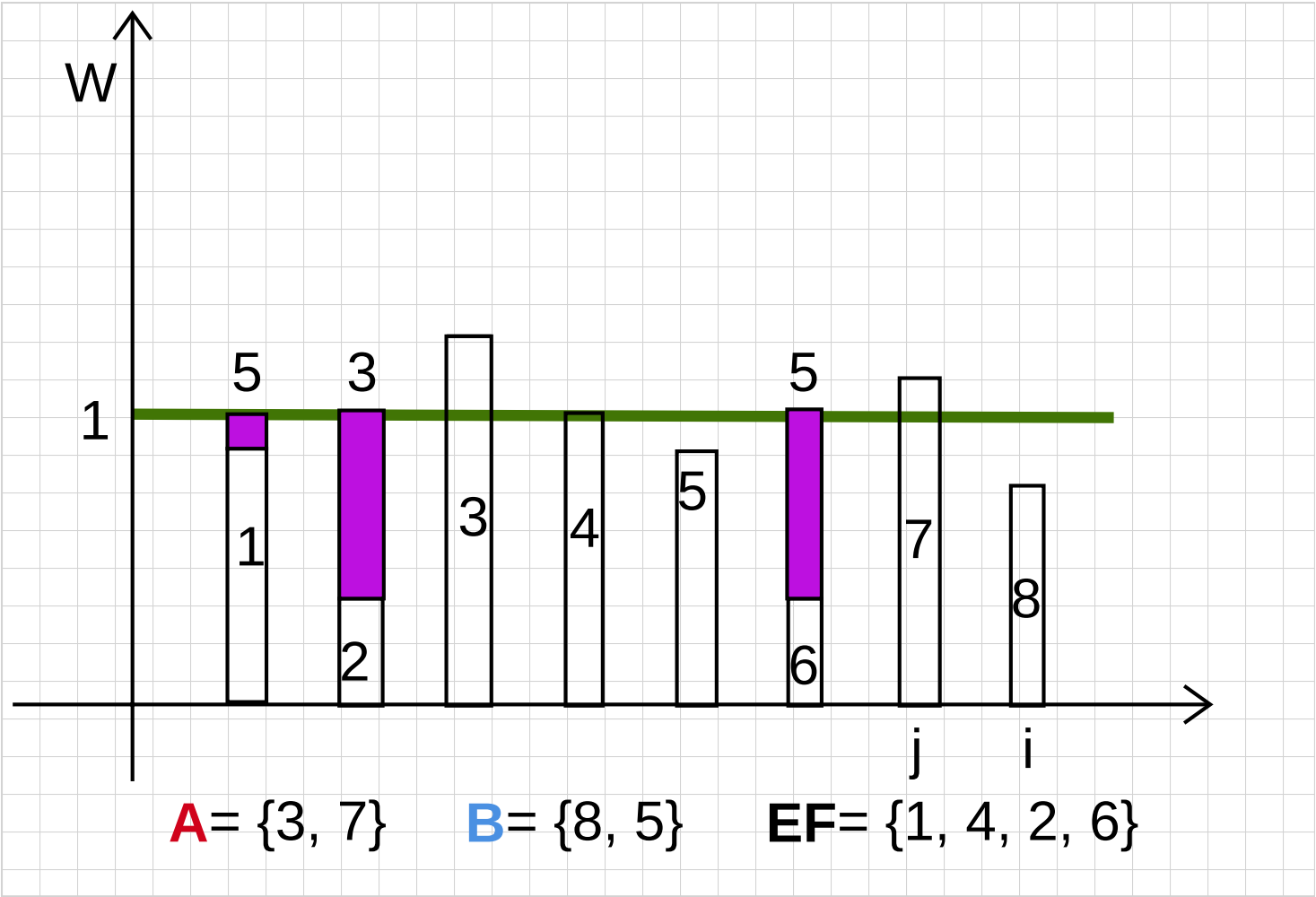
Alias method - Walker's Algorithm



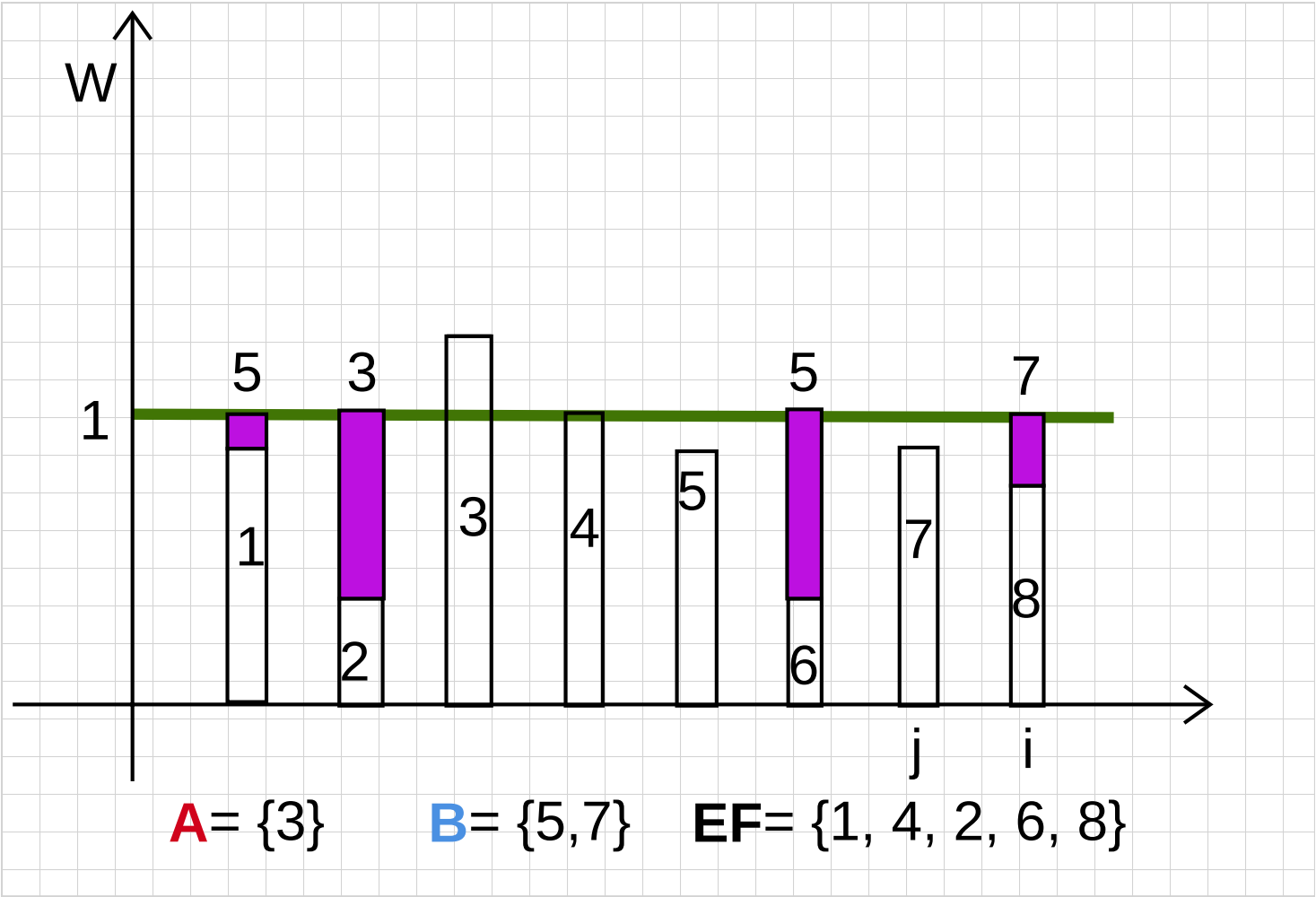
Alias method - Walker's Algorithm



Alias method - Walker's Algorithm



Alias method - Walker's Algorithm



Alias method - Walker's Algorithm

Note that the algorithm is oblivious to the list of values x that takes the random variable. The only important thing is the probability vector p.

The output of the Walker's Algorithm is

- A list of K representing the aliases, where $K[i]$ represent the alias of the i-th value ($K[i] = 0$ if no alias is assigned to i).
- A list of values $r = W/n$, where $W[i]$ represents the value of **white** water in bucket i at the **end of the algorithm**.

We can then represent the original probability vector p as

$$p = \frac{1}{n} \sum_{i=1}^{n} q_i$$

where each q_i is a pmf with

$$q_i(x) = \begin{cases} r_i & \text{if } x = x[i] \\ 1 - r_i & \text{if } x = K[i] \\ 0 & \text{otherwise} \end{cases}$$

References

- A.J. Walker. *New fast method for generating discrete random numbers with arbitrary frequency distributions*. Electronics Letters. 1974
- A.J. Walker. *An Efficient Method for Generating Discrete Random Variables with General Distributions*. ACM Transactions on Mathematical Software. 1977